

TAREA 03 - DOCKER 01 EDWYN LEZAMA RODRÍGUEZ

1. INSTALACIÓN DOCKER

Nos dirigimos a la página oficial Docker y seguimos los pasos a instalar, es decir, los comandos que nos dejan.

Licencia Apache, Versión 2.0. Consulte la [licencia](#) para obtener la licencia completa.

Instalar utilizando el `apt` Repositorio

Antes de instalar Docker Engine por primera vez en una nueva máquina host, usted ¿Necesitas configurar el Docker `apt` Repositorio. Después, puede instalar y actualizar Docker del repositorio.

1. ¡Configurar Docker's `apt` Repositorio.

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Limitaciones de firewall

Requisitos del sistema operativo

Desinstalar versiones antiguas

Métodos de instalación

Instalar utilizando el `apt` Repositorio

Actualizar el motor de Docker

Instalar desde un paquete

Actualizar el motor de Docker

Instalar usando el script de conven

Instale pre-liberaciones

Actualizar Docker después de us

Desinstalar Docker Engine

Próximos pasos

```

dam26@a09eq05:~/prueba_nginx$ cd
dam26@a09eq05:~$ sudo apt update
[sudo] contraseña para dam26:
Lo siento, pruebe otra vez.
[sudo] contraseña para dam26:
Lo siento, pruebe otra vez.
[sudo] contraseña para dam26:
Obj:1 http://deb.debian.org/debian bookworm InRelease
Obj:2 http://security.debian.org/debian-security bookworm-security InRelease
Des:3 http://deb.debian.org/debian bookworm-updates InRelease [55,4 kB]
Des:4 https://download.docker.com/linux/debian bookworm InRelease [46,6 kB]
Descargados 102 kB en 0s (227 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 2 paquetes. Ejecute «apt list --upgradable» para verlos.
dam26@a09eq05:~$ sudo apt install ca-certificates curl
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
ca-certificates ya está en su versión más reciente (20250419-deb12u1).
curl ya está en su versión más reciente (7.88.1-10+deb12u15).
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libfwupd2 libgcb-1.0-0 libsbios-c2 linux-headers-6.1.0-40-amd64 linux-headers-6.1.0-40-common
  linux-headers-6.1.0-41-amd64 linux-headers-6.1.0-41-common linux-headers-6.1.0-42-amd64
  linux-headers-6.1.0-42-common linux-headers-6.1.0-43-amd64 linux-headers-6.1.0-43-common
  linux-headers-6.1.0-44-amd64 linux-headers-6.1.0-44-common linux-headers-6.1.0-47-amd64
  linux-headers-6.1.0-47-common linux-headers-6.1.0-48-amd64 linux-headers-6.1.0-48-common
  linux-image-6.1.0-40-amd64 linux-image-6.1.0-41-amd64 linux-image-6.1.0-42-amd64
  linux-image-6.1.0-43-amd64 linux-image-6.1.0-44-amd64 linux-image-6.1.0-47-amd64
  linux-image-6.1.0-48-amd64 slirp4netns
Utilice «sudo apt autoremove» para eliminarlos.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
dam26@a09eq05:~$ sudo install -m 0755 -d /etc/apt/keyrings
dam26@a09eq05:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
dam26@a09eq05:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
dam26@a09eq05:~$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
> Types: deb
> URIs: https://download.docker.com/linux/ubuntu
> Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
> Components: stable

```

```

> Architectures: $(dpkg --print-architecture)
> Signed-By: /etc/apt/keyrings/docker.asc
> EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: bookworm
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
dam26@a09eq05:~$ sudo apt update
Obj:1 http://security.debian.org/debian-security bookworm-security InRelease
Obj:2 http://deb.debian.org/debian bookworm InRelease
Obj:3 http://deb.debian.org/debian bookworm-updates InRelease
Ign:4 https://download.docker.com/linux/ubuntu bookworm InRelease
Err:5 https://download.docker.com/linux/ubuntu bookworm Release
  404 Not Found [IP: 18.67.250.117 443]
Leyendo lista de paquetes... Hecho
E: El repositorio «https://download.docker.com/linux/ubuntu bookworm Release» no tiene un fichero de Publicación.
N: No se puede actualizar de un repositorio como este de forma segura y por tanto está deshabilitado por omisión.
N: Vea la página de manual apt-secure(8) para los detalles sobre la creación de repositorios y la configuración de usuarios.
dam26@a09eq05:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
docker-ce ya está en su versión más reciente (5:29.8.0-1~debian.12~bookworm).
docker-ce-cli ya está en su versión más reciente (5:29.8.0-1~debian.12~bookworm).
containerd.io ya está en su versión más reciente (2.3.5-1~debian.12~bookworm).
docker-buildx-plugin ya está en su versión más reciente (0.37.0-1~debian.12~bookworm).
docker-compose-plugin ya está en su versión más reciente (5.5.1-1~debian.12~bookworm).
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  libfwupd2 libgcb-1.0-0 libsbios-c2 linux-headers-6.1.0-40-amd64 linux-headers-6.1.0-40-common
  linux-headers-6.1.0-41-amd64 linux-headers-6.1.0-41-common linux-headers-6.1.0-42-amd64
  linux-headers-6.1.0-42-common linux-headers-6.1.0-43-amd64 linux-headers-6.1.0-43-common
  linux-headers-6.1.0-44-amd64 linux-headers-6.1.0-44-common linux-headers-6.1.0-47-amd64
  linux-headers-6.1.0-47-common linux-headers-6.1.0-48-amd64 linux-headers-6.1.0-48-common
  linux-image-6.1.0-40-amd64 linux-image-6.1.0-41-amd64 linux-image-6.1.0-42-amd64
  linux-image-6.1.0-43-amd64 linux-image-6.1.0-44-amd64 linux-image-6.1.0-47-amd64
  linux-image-6.1.0-48-amd64 slirp4netns
Utilice «sudo apt autoremove» para eliminarlos.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
dam26@a09eq05:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine

```

```

0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
dam26@a09eq05:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-09-24 08:42:59 CEST; 2h 52min ago
 TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Main PID: 1769 (dockerd)
         Tasks: 35
        Memory: 150.7M
           CPU: 3.657s
    CGroup: /system.slice/docker.service
            └─ 1769 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
               16578 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8081 -container-ip 172.17.0.2
               16585 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8081 -container-ip 172.17.0.2 -c>

sep 24 10:58:18 a09eq05 dockerd[1769]: time="2026-09-24T10:58:18.294720749+02:00" level=error msg="healthche>
sep 24 10:58:28 a09eq05 dockerd[1769]: time="2026-09-24T10:58:28.300740476+02:00" level=warning msg="healthch>
sep 24 10:58:33 a09eq05 dockerd[1769]: time="2026-09-24T10:58:33.300782664+02:00" level=error msg="healthche>
sep 24 10:58:50 a09eq05 dockerd[1769]: time="2026-09-24T10:58:50.447676432+02:00" level=info msg="sbJoin: gwe>
sep 24 11:04:39 a09eq05 dockerd[1769]: time="2026-09-24T11:04:39.409457294+02:00" level=info msg="received ta>
sep 24 11:08:19 a09eq05 dockerd[1769]: time="2026-09-24T11:08:19.059056429+02:00" level=warning msg="healthch>
sep 24 11:08:19 a09eq05 dockerd[1769]: time="2026-09-24T11:08:19.064206211+02:00" level=warning msg="healthch>
sep 24 11:08:24 a09eq05 dockerd[1769]: time="2026-09-24T11:08:24.059553087+02:00" level=error msg="healthche>
sep 24 11:08:24 a09eq05 dockerd[1769]: time="2026-09-24T11:08:24.064225389+02:00" level=error msg="healthche>
sep 24 11:10:59 a09eq05 dockerd[1769]: time="2026-09-24T11:10:59.473213141+02:00" level=info msg="sbJoin: gwe>
... skipping...
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-09-24 08:42:59 CEST; 2h 52min ago
 TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Main PID: 1769 (dockerd)
         Tasks: 35
        Memory: 150.7M
           CPU: 3.657s
    CGroup: /system.slice/docker.service
            └─ 1769 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
               16578 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8081 -container-ip 172.17.0.2
               16585 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8081 -container-ip 172.17.0.2 -c>

sep 24 10:58:18 a09eq05 dockerd[1769]: time="2026-09-24T10:58:18.294720749+02:00" level=error msg="healthche>

```

```

               16585 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8081 -container-ip 172.17.0.2 -c>

sep 24 10:58:18 a09eq05 dockerd[1769]: time="2026-09-24T10:58:18.294720749+02:00" level=error msg="healthche>
sep 24 10:58:28 a09eq05 dockerd[1769]: time="2026-09-24T10:58:28.300740476+02:00" level=warning msg="healthch>
sep 24 10:58:33 a09eq05 dockerd[1769]: time="2026-09-24T10:58:33.300782664+02:00" level=error msg="healthche>
sep 24 10:58:50 a09eq05 dockerd[1769]: time="2026-09-24T10:58:50.447676432+02:00" level=info msg="sbJoin: gwe>
sep 24 11:04:39 a09eq05 dockerd[1769]: time="2026-09-24T11:04:39.409457294+02:00" level=info msg="received ta>
sep 24 11:08:19 a09eq05 dockerd[1769]: time="2026-09-24T11:08:19.059056429+02:00" level=warning msg="healthch>
sep 24 11:08:19 a09eq05 dockerd[1769]: time="2026-09-24T11:08:19.064206211+02:00" level=warning msg="healthch>
sep 24 11:08:24 a09eq05 dockerd[1769]: time="2026-09-24T11:08:24.059553087+02:00" level=error msg="healthche>
sep 24 11:08:24 a09eq05 dockerd[1769]: time="2026-09-24T11:08:24.064225389+02:00" level=error msg="healthche>
sep 24 11:10:59 a09eq05 dockerd[1769]: time="2026-09-24T11:10:59.473213141+02:00" level=info msg="sbJoin: gwe>
~
~
... skipping...
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-09-24 08:42:59 CEST; 2h 52min ago
 TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Main PID: 1769 (dockerd)
         Tasks: 35
        Memory: 150.7M
           CPU: 3.657s
    CGroup: /system.slice/docker.service
            └─ 1769 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
               16578 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8081 -container-ip 172.17.0.2
               16585 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8081 -container-ip 172.17.0.2 -c>

sep 24 10:58:18 a09eq05 dockerd[1769]: time="2026-09-24T10:58:18.294720749+02:00" level=error msg="healthche>
sep 24 10:58:28 a09eq05 dockerd[1769]: time="2026-09-24T10:58:28.300740476+02:00" level=warning msg="healthch>
sep 24 10:58:33 a09eq05 dockerd[1769]: time="2026-09-24T10:58:33.300782664+02:00" level=error msg="healthche>
sep 24 10:58:50 a09eq05 dockerd[1769]: time="2026-09-24T10:58:50.447676432+02:00" level=info msg="sbJoin: gwe>
sep 24 11:04:39 a09eq05 dockerd[1769]: time="2026-09-24T11:04:39.409457294+02:00" level=info msg="received ta>
sep 24 11:08:19 a09eq05 dockerd[1769]: time="2026-09-24T11:08:19.059056429+02:00" level=warning msg="healthch>
sep 24 11:08:19 a09eq05 dockerd[1769]: time="2026-09-24T11:08:19.064206211+02:00" level=warning msg="healthch>
sep 24 11:08:24 a09eq05 dockerd[1769]: time="2026-09-24T11:08:24.059553087+02:00" level=error msg="healthche>
sep 24 11:08:24 a09eq05 dockerd[1769]: time="2026-09-24T11:08:24.064225389+02:00" level=error msg="healthche>
sep 24 11:10:59 a09eq05 dockerd[1769]: time="2026-09-24T11:10:59.473213141+02:00" level=info msg="sbJoin: gwe>
~
~

```

```
dam26@a09eq05:~$ sudo docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

dam26@a09eq05:~$
```

Siguiente paso descargamos la imagen Alpine https://hub.docker.com/_/alpine y ponemos este código docker pull alpine

```
dam26@a09eq05:~$ docker pull alpine
Using default tag: latest
latest: Pulling from library/alpine
e2de96513ba9: Pull complete
Digest: sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest
dam26@a09eq05:~$
```

Una vez que fijamos la versión, elegimos una desde la web, en mi caso esta: docker pull alpine:3.24.2

```
dam26@a09eq05:~$ status
bash: status: orden no encontrada
dam26@a09eq05:~$ docker pull alpine:3.24.2
3.24.2: Pulling from library/alpine
Digest: sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6
Status: Downloaded newer image for alpine:3.24.2
docker.io/library/alpine:3.24.2
dam26@a09eq05:~$
```

A continuación comprobamos la imagen con el comando: "docker images"

```
~: bash — Konsole
Archivo Editar Ver Marcadores Complementos Preferencias Ayuda
Nueva pestaña Dividir vista Copiar Pegar Buscar
dam26@a09eq05:~/idea-IU-262.10968.63/bin$ cd
dam26@a09eq05:~$ docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
alpine:3.24.2	320994c3b997	8.42MB	0B	
alpine:latest	320994c3b997	8.42MB	0B	
hello-world:latest	74cc54e27dc4	10.1kB	0B	U
imagen_nginx:latest	e9567a94eddc	162MB	0B	U

```
dam26@a09eq05:~$
```

2. CREACIÓN DE CONTENEDOR SIN NOMBRE Y SIN ARRANCAR

Creamos sin nombre y sin arrancarlo. Para ello creamos el contenedor con el comando:
docker create alpine 3.24.2

```
dam26@a09eq05:~$ docker create alpine 3.24.2
964845ec974c6de47583fa7d1612121b3555f4f0ba32b7ffd91bdb74f8dd22ae
dam26@a09eq05:~$
```

Ahora listamos todos los contenedores con “docker ps -a”

```
dam26@a09eq05:~$ docker ps -a

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
964845ec974c	alpine	"3.24.2"	About a minute ago	Created		pedantic_fermat
d058ce4314ce	hello-world	"/hello"	5 days ago	Exited (0) 5 days ago		loving_hellman
88b8588bc586	imagen_nginx	"/docker-entrypoint..."	5 days ago	Exited (0) 4 days ago		wonderful_noyce
4544acd77a29	hello-world	"/hello"	15 months ago	Exited (0) 15 months ago		condescending_knuth

```
dam26@a09eq05:~$
```

Como vemos queda en estado creado “CREATED” y por defecto docker asigna los nombres automáticamente en mi caso le ha asignado “pedantic_fermat”

3. CREACIÓN DE DAM_ALP1 CON UNA SHELL

Creamos y arrancaremos dam_alp1 en una shell, para ello ejecutamos:
docker run -it --name dam_alp1 alpine:3.24.2 sh

```
dam26@a09eq05:~$ docker run -it --name dam_alp1 alpine:3.24.2 sh
/ #
```

¿Qué opciones necesitaremos para escribir adentro? Las siguientes:

-i: Mantiene la entrada estándar abierta para poder mandar comandos

-t: Asigna un pseudo-terminal interactiva

La combinación de ambas, es decir, -it junto con la shell nos permite escribir e interactuar dentro del contenedor

4. CONSULTAR IP DE DAM_ALP1 Y PING A GOOGLE

```
dam26@a09eq05:~$ docker run -it --name dam_alp1 alpine:3.24.2 sh
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if4: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether 16:79:b7:1b:57:ff brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
        valid_lft forever preferred_lft forever
```

La IP de mi contenedor es 172.17.0.2

Ahora haremos ping con google, para ello ejecutamos:

ping -c 4 [google.com](https://www.google.com) (-c4 es para generar 4 respuestas de conexión o sea 4 llamadas para que responda, y así evitamos que siga todo el rato y no lo tengamos que parar de manera manual)

```
/ # ping -c 4 google.com
PING google.com (172.217.171.46): 56 data bytes
64 bytes from 172.217.171.46: seq=0 ttl=114 time=14.267 ms
64 bytes from 172.217.171.46: seq=1 ttl=114 time=21.961 ms
64 bytes from 172.217.171.46: seq=2 ttl=114 time=15.788 ms
64 bytes from 172.217.171.46: seq=3 ttl=114 time=13.841 ms

--- google.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 13.841/16.464/21.961 ms
/ #
```

5. CREACIÓN DE DAM_ALP2, PING UNO A OTRO POR IP Y NOMBRE

Para dejar dam_alp1 funcionando en segundo plano ejecutamos ctrl + P y luego ctrl + Q. A continuación creamos el contenedor dam_alp2, que es como los pasos anteriores similar mismo comando:

docker run -it --name dam_alp2 alpine:3.24.2 sh

```
dam26@a09eq05:~$ docker run -it --name dam_alp2 alpine:3.24.2 sh
/ #
```

Ahora para realizar pings desde dam_alp2 a dam_alp1 ejecutamos ping -c4 (la IP que nos dio antes en dam_alp1)

```
ping -c4 172.17.0.2
```

De otra forma sería poniendo el nombre del contenedor es decir:

```
ping -c4 dam_alp1
```

```
dam26@a09eq05:~$ docker run -it --name dam_alp2 alpine:3.24.2 sh
/ # ping -c4 172.17.0.2
PING 172.17.0.2 (172.17.0.2): 56 data bytes
64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.065 ms
64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.080 ms
64 bytes from 172.17.0.2: seq=2 ttl=64 time=0.090 ms
64 bytes from 172.17.0.2: seq=3 ttl=64 time=0.080 ms

--- 172.17.0.2 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.065/0.078/0.090 ms
/ #
```

```
round-trip min/avg/max = 0.065/0.078/0.090 ms
/ # ping -c4 dam_alp1
ping: bad address 'dam_alp1'
/ #
```

El ping por nombre falla ya que, Docker no cuenta con servicio de resolución DNS automática por nombre de contenedor. Para solucionarlo deberíamos crear una red personalizada mediante `docker network create`

6. CONSUMO DE MEMORIA DE AMBOS CONTENEDORES

Para averiguar el consumo de memoria de ambos en marcha para ello debemos primero desacoplar `dam_apl2` pulsando `ctrl + P` y `ctrl + Q`. Luego ejecutamos:

```
docker stats
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
3f25e838396b	dam_alp2	0.00%	536KiB / 15.39GiB	0.00%	6.78kB / 822B	0B / 8.19kB	1
fe3aea0ab5ad	dam_alp1	0.00%	1.664MiB / 15.39GiB	0.01%	12.8kB / 1.22kB	1.53MB / 8.19kB	1

Este comando muestra métricas en tiempo real sobre el uso de CPU , memoria, red, y disco de todos los contenedores en ejecución

7. SALIDA CON EXIT Y COMPROBACIÓN

Para entrar de nuevo y salir con exit escribimos:

```
docker attach dam_alp1 y luego exit
```

```
dam26@a09eq05:~$ docker attach dam_alp1
/ # exit
dam26@a09eq05:~$
```

Repetimos lo mismo con dam_alp2

```
dam26@a09eq05:~$ docker attach dam_alp2
/ # exit
dam26@a09eq05:~$
```

Ahora comprobamos los estados de los contenedores y volvemos a ejecutar docker stats

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
3f25e838396b	alpine:3.24.2	"sh"	52 minutes ago	Exited (1) 52 seconds ago		dam_alp2
fe3aea0ab5ad	alpine:3.24.2	"sh"	About an hour ago	Exited (0) About a minute ago		dam_alp1
964845ec974c	alpine	"3.24.2"	About an hour ago	Created		pedantic_fermat
d058ce4314ce	hello-world	"/hello"	5 days ago	Exited (0) 5 days ago		loving_hellman
88b8588bc586	imagen_nginx	"/docker-entrypoint..."	5 days ago	Exited (0) 4 days ago		wonderful_noyce
4544acd77a29	hello-world	"/hello"	15 months ago	Exited (0) 15 months ago		condescending_knuth

```
dam26@a09eq05:~$
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
--------------	------	-------	-------------------	-------	---------	-----------	------

Al salir con exit, el proceso principal del contenedor finaliza. Ambos contenedores pasaron al estado Exited (0). Y ahora al ejecutar docker stats , la lista aparece vacía, esto se debe a que únicamente reporta el consumo de recursos de los contenedores en ejecución

8. DISCO OCUPADO Y DISTINCIÓN ENTRE IMÁGENES DE CONTENEDORES

Para calcular el espacio ocupado en disco debemos ejecutar los siguientes comandos:

```
docker system df
```

```
docker ps -as
```

```
^Cdam26@a09eq05:~$ docker system df
```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	4	3	170.5MB	182B (0%)
Containers	6	0	1.167kB	1.167kB (100%)
Local Volumes	0	0	0B	0B
Build Cache	14	0	423B	241B

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES	SIZE
3f25e838396b	alpine:3.24.2	"sh"	About an hour ago	Exited (1) 11 minutes ago		dam_alp2	43B (virtual 8.42MB)
fe3aea0ab5ad	alpine:3.24.2	"sh"	About an hour ago	Exited (0) 12 minutes ago		dam_alp1	31B (virtual 8.42MB)
964845ec974c	alpine	"3.24.2"	About an hour ago	Created		pedantic_fermat	0B (virtual 8.42MB)
d058ce4314ce	hello-world	"/hello"	5 days ago	Exited (0) 5 days ago		loving_hellman	0B (virtual 10.1kB)
88b8588bc586	imagen_nginx	"/docker-entrypoint..."	5 days ago	Exited (0) 4 days ago		wonderful_noyce	1.09kB (virtual 162MB)
4544acd77a29	hello-world	"/hello"	15 months ago	Exited (0) 15 months ago		condescending_knuth	0B (virtual 10.1kB)

El espacio total en imágenes ocupa 170.5MB y los contenedores ocupan menos, unos 1.167kB. La diferencia entre ellos, es que, las imágenes guardan el sistema operativo y las aplicaciones base, ocupan un espacio fijo e inmutable en disco, en cambio los contenedores, añade una capa sola superior ligera donde guardan los cambios y archivos, es debido a eso que su peso propio es de unos bytes.

BIBLIOGRAFÍA DOCKER ALPINE GUÍA DE COMANDOS

<https://jpadillaa.hashnode.dev/docker-comandos-basicos-de-docker>